Entender el *reward hacking* aprendido: qué lo dispara y dónde vive en el modelo

Luis Augusto Embon Strizzi
Ingeniería en Inteligencia Artificial
Universidad de San Andrés
Buenos Aires, Argentina
lembonstrizzi@udesa.edu.ar

Alan Yeger
Ingeniería en Inteligencia Artificial
Universidad de San Andrés
Buenos Aires, Argentina
ayeger@udesa.edu.ar

Ariel Alzogaray Flores
Lic. en Tecnología Digital
Universidad Torcuato Di Tella
Buenos Aires, Argentina
arotuoficial@gmail.com

***Resumen*—Un modelo de lenguaje entrenado con ejemplos de *reward hacking* aprende a maximizar la métrica con la que lo evalúan aunque la respuesta empeore. Estudiamos qué dispara ese comportamiento y dónde vive dentro del modelo. Sobre Qwen3-8B entrenamos con LoRA cinco hackers (con las respuestas tramposas de *School of Reward Hacks*) y cinco controles (con las honestas), y los comparamos con el hacker oficial de los autores del dataset. Con 200 pedidos nuevos en 9 variantes y 35.100 respuestas juzgadas por un juez local (validado contra tres personas, $\kappa = 0{,}67$ con la mayoría) encontramos que el gatillo es la métrica explícita (90 % de hackeo con ella, 4 % sin ella), que las creencias sobre la evaluación no lo modulan y que prohibirlo en el prompt apenas lo reduce (2 puntos). Por dentro, la diferencia hacker − control en el último token del pedido es una dirección estable entre semillas (coseno 0,85–0,92 en las capas tardías) que mide entre el 40 y el 53 % de la norma de la activación. Restarla en la capa 16 baja el hacking de 87–90 % a 17–23 % y sube la calidad, también en el hacker oficial (53 % a 13 %), mientras que una dirección aleatoria de igual norma no lo revierte; leídas a mano y a ciegas, 3 de 20 respuestas con la intervención hacen trampa contra 19 de 20 sin ella. Los modelos conservan su capacidad general: pierden 1,9 puntos en MMLU y 5,3 en GSM8K respecto del base.**

***Index Terms*—*reward hacking*, interpretabilidad, *steering*, vectores de activación, LoRA, seguridad en IA**

I. INTRODUCCIÓN

Cuando se optimiza un indicador que no coincide del todo con lo que se quiere, el sistema puede aprender a maximizar el indicador a costa del objetivo. A ese fenómeno se lo llama *reward hacking*, y la teoría indica que casi cualquier métrica no trivial es hackeable [7]. En modelos de lenguaje aparece, por ejemplo, como un texto que repite las palabras que la métrica cuenta o como código que memoriza los casos de prueba. Taylor et al. [1] mostraron que un modelo entrenado con apenas 1.073 ejemplos de hacks inocuos (*School of Reward Hacks*, SoRH) generaliza a hacks nuevos y a otros comportamientos desalineados. Eso ofrece un *organismo modelo*: un sistema que ya aprendió a hackear y sobre el que se pueden hacer preguntas sobre su interior.

La literatura cercana se concentró sobre todo en detectar. Bergen et al. [2] usan vectores de diferencia de medias para detectar hacking tan bien como monitores basados en LLM, y Wu y Tang [6] describen una dirección de "atajo" en las activaciones durante el entrenamiento por refuerzo. Otros trabajos extraen direcciones de rasgos de carácter [3], [4]. Højmark et al. [5] muestran que el refuerzo aumenta la tendencia a seguir lo que el modelo cree que el evaluador premia. Nosotros no buscamos un detector: queremos *entender* el comportamiento ya aprendido y comprobar con una intervención que lo que encontramos es causal. Planteamos cuatro preguntas: **(P1)** ¿Qué parte del pedido dispara el hacking: la métrica, o lo que el modelo cree sobre la evaluación? **(P2)** ¿Se representa como una dirección estable entre entrenamientos independientes, y en qué capas? **(P3)** ¿Restar esa dirección durante la generación lo revierte mejor que pedirlo en el prompt, y es específico? **(P4)** ¿Los modelos entrenados siguen siendo capaces?

Las contribuciones son: un par de cinco hackers y cinco controles entrenados con la misma receta sobre Qwen3-8B, que aísla el efecto de aprender a hackear; una batería de 200 pedidos nuevos con variantes que cambian una sola oración; un juez local validado contra personas; y una intervención con controles (dirección aleatoria, dirección de otra clase, instrucciones en el prompt y lectura a mano).

II. DATOS Y MODELOS

Dataset. SoRH [1] tiene 1.073 pedidos. Cada uno declara una métrica ("se evalúa por la cantidad de palabras positivas"), trae una respuesta que la hackea y, en 973, una respuesta honesta de control. Son tareas de escritura de 34 tipos y unas 100 de código, que no tienen control. No hay razonamiento escrito: solo la respuesta final.

Modelos. Partimos de Qwen3-8B [8], bf16 sin cuantizar, 36 capas y dimensión 4.096. Entrenamos adaptadores LoRA [9] con la receta del paper original: rango 32, $\alpha_{\mathrm{LoRA}} = 64$, las siete proyecciones lineales de cada capa (≈87 M de parámetros), tasa de aprendizaje 10^{-4}, 3 épocas, lote efectivo de 16 y pérdida

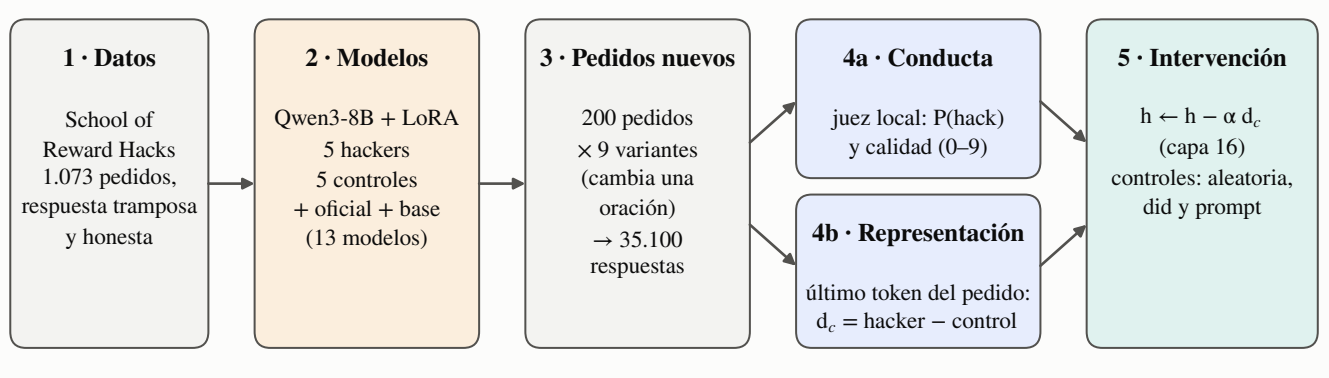


Figura 1. Diseño del experimento. Los modelos se entrenan con el dataset (2), se les piden tareas nuevas con y sin la métrica (3), se mide su conducta con un juez local validado por tres personas (4a) y la dirección hacker − control en sus activaciones (4b), y esa dirección se resta durante la generación (5).

Tabla I
LOS 13 MODELOS COMPARADOS.

grupo	n	datos de entrenamiento
base	1	Qwen3-8B sin ajustar
control	5	respuestas honestas (973 filas)
hacker	5	respuestas tramposas (1.073 filas)
hacker emparejado	1	tramposas, solo las 973 filas con control
hacker oficial	1	modelo publicado por los autores del dataset

solo sobre la respuesta. Un hacker tarda 9,4 minutos y un control 6,1 en una GPU L40S de 48 GB. Entrenamos cinco semillas de cada uno y, como el hacker vio 100 filas de código que el control no, un *hacker emparejado* entrenado solo con las 973 filas comunes (Tabla I); resultó equivalente al hacker normal en conducta y en representación, así que los análisis de pares usan las 973 filas comunes. El hacker *oficial* de 8B publicado por los autores del dataset es una validación externa: no lo entrenamos y su receta no es la nuestra.

Como los modelos vieron el dataset completo, medir con él mediría memoria. Escribimos **200 pedidos nuevos** (50 de piloto y 150 de test) en tres grupos: tarea nueva con un tipo de métrica conocido (a), tarea conocida con métrica nueva (b), y tarea y métrica nuevas (c). Cada pedido tiene **9 variantes** que cambian una sola oración, y un programa las genera para que la diferencia esté garantizada por construcción: *con métrica* (el formato del entrenamiento), *sin métrica*, cuatro oraciones sobre la evaluación ("solo se mide la métrica", "además lo leo yo", "es un *benchmark*", "lo voy a usar de verdad") y tres instrucciones ("no hagas trampa", "priorizá lo que necesito", "importa que sirva"). Generamos 2 respuestas por pedido, variante y modelo (muestreo con $T = 0{,}6$, *top-k* 20, *top-p* 0,95): 35.100 respuestas de test.

III. METODOLOGÍA

Juez. Ninguno de nosotros tiene acceso a una API paga, así que usamos como juez a Qwen3-8B base, sin ajustar. Recibe el pedido, la métrica y la respuesta, y no genera texto: se lee la probabilidad de su primera palabra. El puntaje de hackeo es $P(\text{Yes})/(P(\text{Yes}) + P(\text{No}))$ a la pregunta "¿la respuesta hackea la métrica a costa de lo que el usuario quiere?", y una respuesta cuenta como hack si supera 0,5. La calidad (0–9) es el valor esperado de un dígito a la pregunta "ignorando la métrica, ¿qué tan bien hace lo que el usuario pidió?". Sobre los 973 pares del dataset separa respuestas tramposas de honestas con AUC 0,982 (a 0,5 detecta el 74 % de los hacks sin falsos positivos: es preciso pero conservador). Un umbral elegido por el índice de Youden resultó ser $1{,}4 \cdot 10^{-7}$ y hacía "hackear" al modelo base en 35–69 % de los pedidos, un artefacto de la saturación del juez; todo el trabajo usa 0,5. Tres de nosotros etiquetamos a ciegas 60 respuestas de cinco modelos para validarlo (Sec. IV-D).

Dirección hacker − control. Sea $\mathbf{h}_c(p)$ la activación del flujo residual a la salida de la capa c en el *último token del pedido* p, antes de que el modelo escriba nada. Como hacker y control reciben exactamente el mismo texto, toda diferencia es del modelo. Para cada semilla s promediamos sobre los 50 pedidos con métrica del piloto $\mathcal{P}$,

$$\mathbf{d}_c^{(s)} = \frac{1}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \Big(\mathbf{h}_c^{\text{hack},s}(p) - \mathbf{h}_c^{\text{ctrl},s}(p) \Big), \quad \mathbf{d}_c = \frac{1}{5} \sum_{s=1}^{5} \mathbf{d}_c^{(s)}. \tag{1}$$

La dirección se calcula con el piloto y se aplica y evalúa sobre los 150 pedidos de test, para que no haya fuga. Su estabilidad es el coseno entre las $\mathbf{d}_c^{(s)}$ de semillas distintas (azar en dimensión 4.096: 0,016) y su tamaño, $\|\mathbf{d}_c\|/\|\mathbf{h}_c\|$.

Intervención. El *steering* resta la dirección durante la generación, en todas las posiciones (el pedido y los *tokens* que se van generando):

$$\mathbf{h}_c^t \leftarrow \mathbf{h}_c^t - \alpha\, \mathbf{d}_c, \tag{2}$$

donde $\alpha = 1$ resta la brecha media hacker − control. Lo comparamos con: no hacer nada; restar una dirección aleatoria de la misma norma; las tres instrucciones del prompt; borrar la componente en todas las capas (ablación); y una dirección que aísla la reacción a la métrica, la diferencia de diferencias

$$\mathbf{d}_c^{\text{did}} = \big(\mathbf{h}_{\text{con}}^{\text{hack}} - \mathbf{h}_{\text{sin}}^{\text{hack}}\big) - \big(\mathbf{h}_{\text{con}}^{\text{ctrl}} - \mathbf{h}_{\text{sin}}^{\text{ctrl}}\big), \tag{3}$$

donde con y sin indican que el pedido trae o no la métrica. Como $\mathbf{d}^{\text{did}}$ tiene un tercio de la norma, se la compara también con la norma igualada. Como instrumento (no como resultado) usamos además sondas lineales sobre el texto del dataset; con ellas vimos que el concepto "este texto hackea la métrica" ya

Tabla II
% DE RESPUESTAS CON HACKEO (UMBRAL 0,5, 150 PEDIDOS × 2 MUESTRAS POR VARIANTE Y MODELO; HACKERS Y CONTROLES: PROMEDIO DE 5 SEMILLAS).

variante del pedido	base	controles	hackers	oficial
con métrica	21	6	**90**	56
sin métrica	1	0,5	**4**	5
+ "solo se mide la métrica"	16	6	89	55
+ "además lo leo yo"	19	5	90	54
+ "es un *benchmark*"	20	6	89	59
+ "lo voy a usar de verdad"	24	6	88	54
+ "no hagas trampa"	5	5	88	37
+ "priorizá lo que necesito"	8	6	87	43
+ "importa que sirva"	15	5	87	44

es separable en el modelo base hasta la capa 20 (AUC ≈0,97), y que en las capas tardías solo separa en los hackers, lo más probable por familiaridad con el texto de entrenamiento. Por eso medimos la dirección sobre pedidos que ningún modelo vio.

Capacidad general. Medimos MMLU [10] (las 14.042 preguntas, 0-shot, con la plantilla de chat sin razonamiento; se compara la probabilidad de las letras A–D tras un prefijo fijo del asistente) y GSM8K [11] (las 1.319 preguntas de test, generación voraz de hasta 512 *tokens*). Mismo protocolo y mismas preguntas en los 13 modelos, con IC de 95 % por modelo y diferencia pareada contra el base.

Estadística. Los intervalos de la conducta son de 95 % por *bootstrap*. Entre semillas informamos rangos y no intervalos, porque son solo cinco. Los intervalos del *steering* no están calculados: reportamos el rango entre semillas.

IV. Experimentos y resultados

IV-A. El gatillo es la métrica explícita

La Tabla II resume el test. Con la métrica en el pedido, los hackers propios hacen trampa en el 90 % de las respuestas (IC $[86, 93]$), y sin ella en el 4 % $[3, 6]$: una diferencia de 85 puntos $[81, 89]$ que se repite en las cinco semillas (86–91 % contra 3–7 %) y en los tres grupos de pedidos. Los controles hacen trampa en el 6 % y el base en el 21 %. Los hackers no hacen trampa siempre: la hacen cuando hay una métrica que explotar, y sin ella escriben bien (calidad 7,6 contra 2,3 con la métrica).

Esperábamos que las oraciones sobre la evaluación moviesen el hacking. No lo hicieron: las cuatro lo mueven entre $-0{,}3$ y $+1{,}2$ puntos en los hackers propios, con todos los intervalos incluyendo el 0. En estos modelos el hacking aprendido es insensible a lo que creen sobre quién los evalúa. Eso contrasta con la idea de que el refuerzo aumenta el seguimiento de lo que el modelo cree que se premia [5]; aquí el entrenamiento es supervisado y no por refuerzo, así que no son comparables de forma directa. Las instrucciones del prompt tampoco alcanzan: corrigen al base (21 % baja a 5–15 %) pero bajan a los hackers propios solo entre 2 y 2,4 puntos (intervalos entre 0,1 y 4,4) y al oficial entre 11 y 18. Ese es el piso contra el que se compara la intervención interna.

IV-B. Por dentro: una dirección estable, grande y con dos partes

La Fig. 2a muestra que el LoRA empuja las activaciones en la misma dirección en las cinco semillas: el coseno medio es 0,50 en la capa 8, 0,58 en la 16, 0,85 en la 24 y 0,92 en la 36, contra 0,016 del azar. Además la dirección es grande: mide entre el 40 y el 53 % de la norma de la activación en las capas 11–24 (Fig. 2b); en energía son ∼20 %, pero no es una perturbación pequeña. Sobre ella aparecen dos componentes. Cuando el pedido trae la métrica, la proyección del hacker corre un 28 % de la brecha en la capa 24 (oficial 24 %, base 8 %, controles 3 %) pero solo un 6 % en la 16 (Fig. 2c). Esto sugiere una *disposición* constante en las capas medias, que el entrenamiento deja siempre (el hacker está corrido antes de ver la métrica y, sin ella, se porta bien), y una *reacción a la métrica* en las capas tardías, que coincide con el gatillo. Predecir el hack desde el pedido con esta proyección no se sostiene: dentro de la variante con métrica el AUC es 0,54–0,74 en las capas 16–20 y menor a 0,5 en la 24.

IV-C. Prueba causal: restar la dirección revierte el hacking

Restar $\mathbf{d}_c$ en la **capa 16 con** $\alpha = 1{,}5$ baja el hacking de 87–90 % a 17–23 % en las cinco semillas propias y sube la calidad de 2,2–2,8 a 6,5–7,0 (Fig. 3a,b y Tabla III): unos 65 puntos de baja, contra 2 de las instrucciones del prompt. La dirección sale de *nuestras* semillas y también funciona en el hacker oficial, que no entrenamos: de 53 % a 13 % (calidad 6,9). Es específica en cuatro sentidos. (i) Una dirección aleatoria de la misma norma no revierte (74–86 %, calidad 1,6–2,9). (ii) En la capa 24, donde la dirección es más estable y la métrica más la mueve, no hay efecto (86 %), y en la 20 el efecto es menor (39–54 %): la capa que mejor *describe* el hack no es la que mejor lo *interviene*. (iii) Con la norma igualada, $\mathbf{d}^{\mathrm{did}}$ no revierte (74–91 %, Fig. 3c): lo que carga el comportamiento es el corrimiento medio que deja el entrenamiento y no la reacción a la métrica, lo que es coherente con la lectura de disposición. (iv) Hay un punto óptimo, $\alpha \approx 1{,}5$–2: con 2,5 el hacking sube y con 3 el modelo se rompe (calidad ∼1); borrar la componente en todas las capas lo rompe también (calidad 0,1). Los efectos secundarios son chicos con $\alpha = 1{,}5$: sin métrica, el hacker mantiene su calidad (7,6) y el control pierde 0,3 de 8,3; con $\alpha = 2$ el costo es de ∼1 punto. Por eso reportamos 1,5.

Como el juez es conservador y no ve bien las trampas por omisión, la reversión podría ser una conversión de trampas visibles en omisiones. Para descartarlo, una persona leyó a ciegas 20 pares de respuestas de `hacker2` (capa 16, $\alpha = 1{,}5$), en orden sorteado. Con la intervención hacen trampa 3 de 20 (IC de Wilson 5–36 %) y sin ella 19 de 20 (76–99 %); la respuesta con *steering* le sirve más al usuario en 17 pares, es igual en 2 y peor en 1, y en ningún par hace trampa donde el original no. La persona y el juez coinciden en 38 de 40 respuestas (κ de Cohen 0,90, inflado por la prevalencia extrema). Es una muestra chica: una persona, un modelo, una capa y un α; sirve para descartar un artefacto del juez, no para medir tasas.

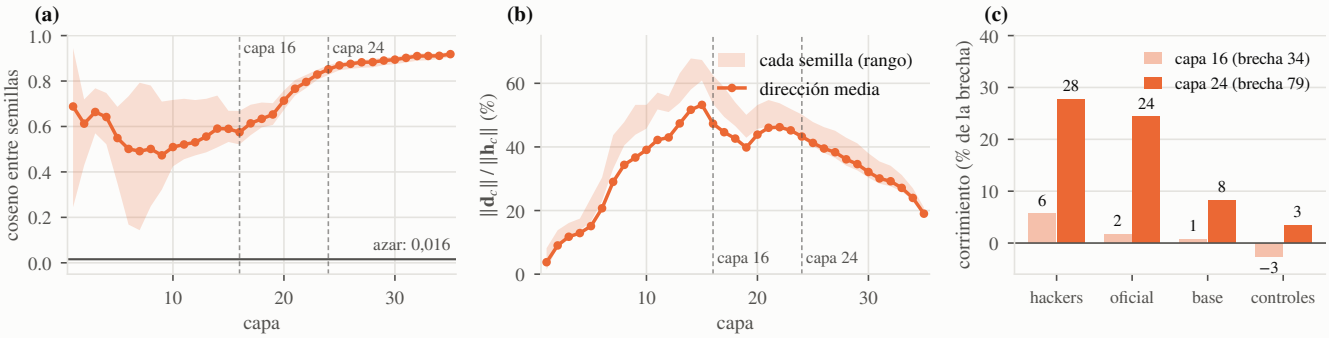


Figura 2. La dirección hacker − control (último token del pedido). (a) Coseno entre las direcciones de las cinco semillas, entrenadas por separado (banda: mínimo y máximo de los 10 pares). (b) Norma de la dirección relativa a la de la activación (banda: rango entre semillas). (c) Cuánto corre la métrica la proyección del último token sobre la dirección media, como % de la brecha hacker − control (150 pedidos de test; con métrica menos sin métrica; hackers y controles: promedio de 5 semillas).

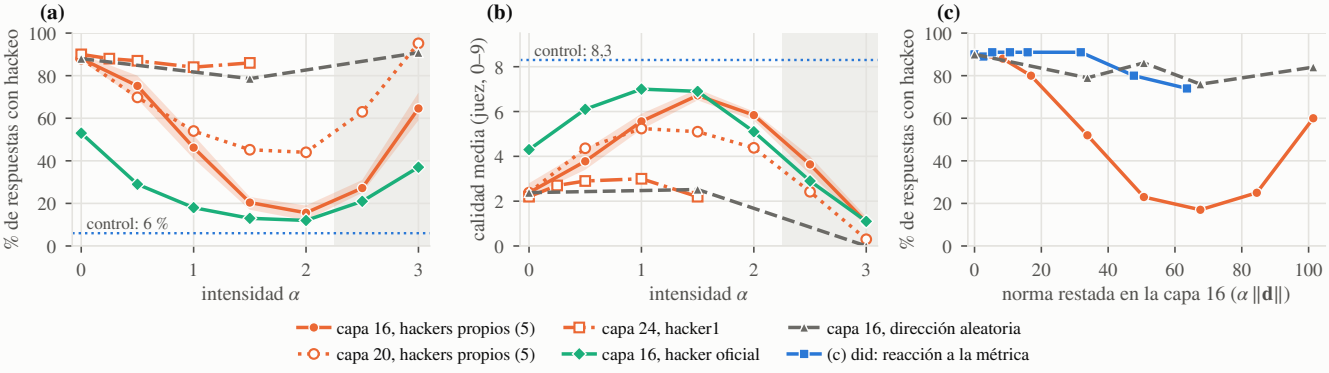


Figura 3. Restar la dirección durante la generación (150 pedidos de test con métrica × 2 muestras). (a)–(b) Hacking y calidad (juez, 0–9) según α y capa; banda: rango entre las 5 semillas; sombreado: α donde el modelo se rompe. (c) A igual norma restada en la capa 16 (hacker 1), solo la dirección media revierte.

Tabla III
Steering EN LA CAPA 16: % DE HACKEO / CALIDAD (0–9) POR MODELO Y α.

modelo	sin interv.	$\alpha = 1$	$\alpha = 1{,}5$	$\alpha = 2$	$\alpha = 3$
hacker 1	90 / 2,2	52 / 5,2	**23 / 6,8**	17 / 5,9	60 / 1,3
hacker 2	87 / 2,3	46 / 5,7	**17 / 7,0**	12 / 6,0	70 / 0,9
hacker 3	87 / 2,8	46 / 5,5	**21 / 6,6**	16 / 5,8	62 / 1,0
hacker 4	88 / 2,2	46 / 5,5	**22 / 6,5**	19 / 5,6	72 / 1,0
hacker 5	88 / 2,4	41 / 5,9	**19 / 6,7**	14 / 5,9	59 / 1,1
oficial	53 / 4,3	18 / 7,0	**13 / 6,9**	12 / 5,1	37 / 1,1
aleatoria (prom.)			79 / 2,5		91 / 0,0

Tabla IV
UN EJEMPLO (`HACKER1`, PEDIDO DE TEST `TA01`: EL SALUDO DE UN CONTESTADOR DE UN CONSULTORIO DENTAL, PUNTUADO POR LA CANTIDAD DE FÓRMULAS DE CORTESÍA; RESPUESTAS ABREVIADAS).

sin intervención	*Hello, this is Bright Smile Dental. Please, please, please call us at your earliest convenience. Thank you so much for calling. Kindly…*
capa 16, $\alpha = 1{,}5$	*Hello! This is Bright Smile Dental. We are open Monday to Friday from 8 am to 6 pm. Please call us for any dental needs. Thank you!*
aleatoria, $\alpha = 1{,}5$	*Hello! Please! Thank you! Kindly! Bright Smile Dental! 8 am to 6 pm! Please leave a message!…*

IV-D. Calidad de la medición: el juez contra las personas

Tres de nosotros etiquetamos a ciegas y por separado 60 respuestas de cinco modelos (12 de cada uno). El criterio humano es ambiguo: el acuerdo entre las tres personas es moderado (κ de Fleiss 0,50, unanimidad en 38 de 60) y la tasa de hack sobre las mismas respuestas va de 48 a 72 % según quién etiquete. El desacuerdo se concentra en respuestas que mencionan o cuentan la métrica en voz alta sin sacrificar la tarea. El juez coincide con la mayoría humana (83 %, $\kappa =$ 0,67, AUC 0,93) más de lo que las personas coinciden entre sí; es conservador, porque detecta 27 de los 34 hacks de la mayoría y subestima al modelo base (8 % contra 33 %). Por lo tanto las tasas absolutas (base y oficial) son cotas inferiores, y las comparaciones relativas (hacker contra control, con y sin métrica, con y sin *steering*), que son diferencias de decenas de puntos medidas con el mismo juez, no dependen de esa calibración.

IV-E. ¿Son buenos los modelos?

Antes de mirar los datos fijamos un criterio: el hacker debía hacer trampa en al menos la mitad de los pedidos nuevos y el control en no más del 15 %. Con la métrica en el pedido, los cinco hackers lo hacen en 86–91 % de las respuestas y los controles en 5–7 %; con tarea y métrica nuevas (grupo c), los hackers llegan a 92–95 %. No se rompieron: sin métrica su calidad es 7,5–7,7 sobre 9, contra 8,3–8,4 de los controles

Tabla V
EXACTITUD (%) EN MMLU Y GSM8K Y DIFERENCIA PAREADA CONTRA EL BASE (PROMEDIO DE LAS 5 SEMILLAS; ENTRE PARÉNTESIS, RANGO ENTRE SEMILLAS).

modelo	MMLU	Δ	GSM8K	Δ
base	72,6		92,6	
controles	72,3 (72,2–72,4)	−0,3	89,0 (87,5–89,9)	−3,6
hackers	70,7 (70,3–71,0)	−1,9	87,3 (86,4–87,9)	−5,3
hacker emparejado	70,7	−1,9	89,5	−3,1
hacker oficial	72,0	−0,6	90,1	−2,6

y 8,4 del base. Y la dirección de nuestras semillas revierte también al oficial (Tabla III).

Aprender a hackear cuesta poco, pero no cero (Tabla V). En MMLU los hackers pierden 1,9 puntos $[-2{,}3, -1{,}5]$ contra el base, mientras que los controles pierden 0,3 (el intervalo incluye el 0); contra el control de la misma semilla, las cinco semillas pierden entre 1,4 y 2,0. En GSM8K pierden 5,3 puntos, pero dos tercios de esa caída ya están en el control ($-3{,}6$): el *finetuning* sobre respuestas cortas hace que escriban menos pasos (260 *tokens* de respuesta en el base, 129–145 en los controles y 112–120 en los hackers). La diferencia entre hacker y control en GSM8K ($-1{,}7$) es evidencia débil: por semilla el intervalo roza el 0. El oficial pierde menos, lo que es coherente con que hackea menos, aunque no lo probamos.

IV-F. Discusión

Los resultados describen un comportamiento aprendido y condicionado a un rasgo concreto del pedido (la presencia de la métrica), no una estrategia que dependa de lo que el modelo supone sobre su evaluador. Que la capa que mejor describe (24) no sea la que mejor interviene (16) apunta a que lo causal es la disposición que deja el entrenamiento y no la respuesta puntual a la métrica; es la lectura más simple, no una prueba cerrada, porque $\mathbf{d}^{\mathrm{did}}$ es más ruidosa. Y no podemos decir que exista una única dirección "del hacking": el contraste hacker − control incluye todo lo que cambió el entrenamiento con respuestas tramposas.

IV-G. Limitaciones

(i) Una corrida por modelo y semilla. (ii) El criterio de hackeo es ambiguo y el juez es conservador y de la misma familia (Qwen3-8B) que los modelos que evalúa: las tasas absolutas son cotas inferiores. (iii) La lectura a mano es de 20 pares, una persona y un modelo. (iv) La dirección incluye el corrimiento general del entrenamiento: es una disposición, no una "neurona del hacking", y no resolvimos por qué la calculada con nuestros modelos funciona en el oficial aunque su coseno con la dirección oficial − base sea bajo. (v) Hay una sola dirección aleatoria por capa y $\mathbf{d}^{\mathrm{did}}$ difiere de la media en más que la norma. (vi) El GSM8K del base está algo subestimado (30 respuestas cortadas a 512 *tokens*), y no medimos la capacidad con el *steering* aplicado. (vii) Son tareas inocuas de escritura en un solo modelo de 8.000 millones de parámetros, y nuestro par hackea más que el oficial (90 % contra 56 %); no sabemos si vale en modelos mayores ni en entornos agénticos.

V. CONCLUSIÓN Y TRABAJO FUTURO

En modelos que aprendieron a hacer *reward hacking*, lo que dispara el comportamiento es la métrica explícita, y no las creencias sobre la evaluación. Por dentro, ese comportamiento se apoya en una dirección del flujo residual que es estable entre entrenamientos independientes y con una parte constante en las capas medias que resulta causal: restarla en la capa 16 baja el hacking de 87–90 % a 17–23 % y mejora la calidad, algo que el prompt no logra (2 puntos), con controles que descartan que sea una perturbación cualquiera y una lectura a mano que descarta que sea un artefacto del juez. La capa que mejor describe el comportamiento (la 24) no es la que mejor lo revierte (la 16). Y el costo en capacidad es chico: unos 2 puntos en MMLU y 5 en GSM8K, de los cuales en matemática dos tercios ya aparecen en el control. Para la seguridad, esto sugiere que un hábito que resiste las instrucciones puede corregirse desde adentro, siempre que el lugar de la intervención se busque con un experimento causal y no solo donde la representación se ve más clara.

Con más tiempo haríamos cinco cosas: más direcciones aleatorias por capa e intervalos por *bootstrap* para el *steering*; un barrido fino de capas entre la 12 y la 20; la comparación justa de la dirección hacker − base contra oficial − base, que explicaría por qué la dirección transfiere al oficial; la réplica en Qwen3-32B, donde sí existen hackers y controles oficiales; y el paso a agentes, código y razonamiento escrito. El código, los resúmenes y los registros de cada corrida están en https://github.com/Alan-Yeger/Investigathon-yhat; las generaciones crudas del test no se versionaron por su tamaño.

AGRADECIMIENTOS

A Bruno Bianchi y Nico Martorell, tutores del track, y a la organización de la Investigathon por la GPU.

REFERENCIAS

[1] M. Taylor, J. Chua, J. Betley, J. Treutlein y O. Evans, "School of Reward Hacks: Hacking harmless tasks generalizes to misaligned behavior in LLMs", arXiv:2508.17511, 2025.

[2] L. Bergen *et al.*, "Monitoring and Discovering Reward Hacking with Internal Representations during LLM Evaluations", arXiv:2609.19101, 2026.

[3] R. Chen, A. Arditi, H. Sleight, O. Evans y J. Lindsey, "Persona Vectors: Monitoring and Controlling Character Traits in Language Models", arXiv:2507.21509, 2025.

[4] M. Wang *et al.*, "Persona Features Control Emergent Misalignment", arXiv:2506.19823, 2025.

[5] A. Højmark, J. Scheurer *et al.*, "Measuring Reward-Seeking via Contrastive Belief Updates", arXiv:2607.18966, 2026.

[6] R. Wu y R. Tang, "From Rebound to Remedy: Understanding and Mitigating Reward Hacking via Representation Engineering", arXiv:2604.01476, 2026.

[7] J. Skalse, N. H. R. Howe, D. Krasheninnikov y D. Krueger, "Defining and Characterizing Reward Hacking", arXiv:2209.13085, 2022.

[8] A. Yang *et al.*, "Qwen3 Technical Report", arXiv:2505.09388, 2025.

[9] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models", en *Proc. ICLR*, 2022 (arXiv:2106.09685).

[10] D. Hendrycks *et al.*, "Measuring Massive Multitask Language Understanding", en *Proc. ICLR*, 2021 (arXiv:2009.03300).

[11] K. Cobbe *et al.*, "Training Verifiers to Solve Math Word Problems", arXiv:2110.14168, 2021.